

Predicting the Unpredictable: Innovations in Volcanic Eruption Forecasting

Eurylyn B. Florido
FAITH Colleges
s2023101192@firstasia.edu.ph

John Mitchell M. Fruto
FAITH Colleges
s2023101186@firstasia.edu.ph

Marco A. Burdeos
FAITH Colleges
ma.burdeos@firstasia.edu.ph

Abstract - This paper talks about a Python project designed to predict when volcanoes might erupt by using datasets. The project uses Python's strong tools to make a system that looks at old volcanic data and environment details to predict eruptions. With libraries like Pandas, NumPy, Scikit-learn, TensorFlow, and tools like Flask, it tries to give a fast, easy, and scalable solution. Some main features are real-time data handling, machine learning models, user-friendly design, and data pictures. The results shows that it predicts eruptions with good accuracy, showing it can help early warning systems. This project aims to improve disaster planning and lower volcano risks.

Index Terms - Python Programming, Volcanic Eruption Prediction, Real-time Data Processing, Machine Learning Models, Data Visualization, Flask, Pandas, NumPy, Scikit-learn, TensorFlow, Automation, Disaster Preparedness, Environmental Monitoring.

INTRODUCTION

[2] Volcanoes are amazing but dangerous. Their eruptions can destroy towns, block air travel, and harm the environment for a long time, earning them the name "Beautiful Disasters." Predicting when a volcano will erupt is still one of the hardest problems in earth science. Even though monitoring systems have improved over the years, it is still extremely hard to predict eruptions accurately. Mistakes in predicting the time or power of an eruption can cause major loss of life or unnecessary evacuations.

Traditional methods for predicting eruptions look at ground changes, gas releases, and earthquakes. While these give useful information, they are limited because the data is hard to handle and understand. Even with better tools, scientists might miss key warning signs or mistake

normal volcanic activity for a threat because volcanoes are so unpredictable.

This project suggests a new way to forecast eruptions using machine learning. Machine learning is great at spotting patterns in big data, even when the patterns are small or complicated. By using these tools on volcanic data, the project aims to make predictions more accurate and give early warnings to at-risk communities.

METHODOLOGY

I. Requirements

This project needs Python and some libraries for working with data, making visuals, and machine learning. It uses NumPy and Pandas to handle data, and Plotly to make interactive graphs. For machine learning, it uses Scikit-learn for choosing and testing models, plus advanced tools like LightGBM, XGBoost, and TensorFlow/Keras to build and improve prediction models. Optuna is also used to fine-tune model settings. The data comes from CSV files, so the setup must include these files and the right folders. To run this project, you need to know Python, data cleaning, and machine learning methods.

II. Pseudocode

The program's logic follows a structured flow, as illustrated in the pseudocode:

- START
 - 1. IMPORT necessary libraries (numpy, pandas, plotly, sklearn, lightgbm, xgboost, tensorflow, etc.)

2. Load and Prepare Data
 - LOAD training dataset from CSV file
 - LOAD sample submission file
3. Exploratory Data Analysis (EDA)
 - DISPLAY summary statistics of the dataset
 - PLOT histograms and other visualizations to understand data distributions
4. Feature Selection
 - SELECT important features using methods like Recursive Feature Elimination (RFE) or correlation analysis
5. Data Preprocessing
 - SPLIT data into training and validation sets using train_test_split or KFold
 - NORMALIZE or standardize features if required
 - HANDLE missing values or outliers if necessary
6. Model Training
 - DEFINE models (e.g., DecisionTreeRegressor, LGBMRegressor, XGBRegressor, Neural Networks)
 - SET hyperparameters for each model
 - TRAIN models using training data
 - VALIDATE models on validation data
7. Hyperparameter Optimization
 - USE Optuna for hyperparameter tuning
 - DEFINE objective function for optimization (e.g., minimizing validation loss)
 - RUN optimization loop
8. Evaluate Models
 - CALCULATE performance metrics (e.g., mean squared error)
 - COMPARE models to select the best-performing one
9. Final Model Training
 - TRAIN the best model on the entire training dataset
10. Predictions
 - LOAD test dataset
 - MAKE predictions using the trained model
 - SAVE predictions to submission file
11. Save and Document Results
 - EXPORT trained model and important metrics
 - GENERATE plots for analysis and reporting

- END

Figure 1.

This pseudocode explains how to predict volcanic eruptions. First, it start by importing important libraries like numpy, pandas, plotly, sklearn, LightGBM, XGBoost, and tensorflow to handle data, make visuals, build models, and optimization. Then, the training data is loaded, and exploratory data analysis (EDA) is done to find patterns, check statistics, and creating graphs. Key features are chosen using methods like Recursive Feature Elimination (RFE) or correlation analysis. The data is prepared by splitting into training and validation sets, normalizing values, and fix missing parts. Finally, reports is created to ensure everything is reproducible.

III. Flowchart

To visually represent the program's logic, a flowchart was developed to illustrate the program's processes, from input validation to output generation. This enhances the clarity of the program and facilitates its maintenance and future improvements.

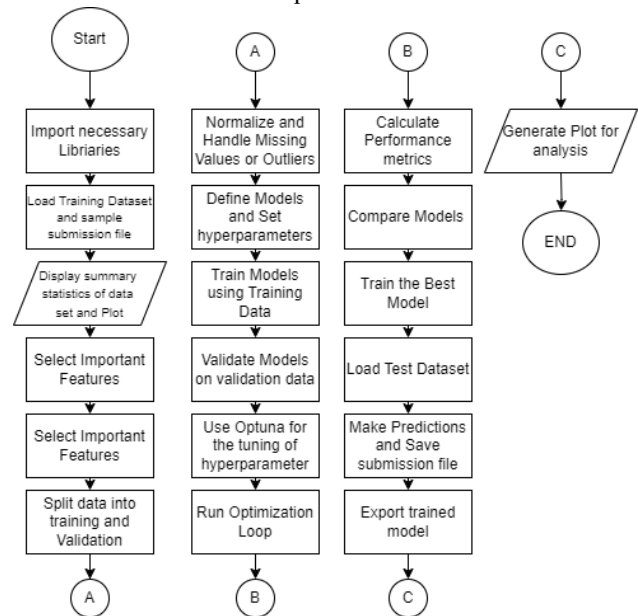


Figure 1.

The program starts by importing the necessary libraries for data handling, creating visuals, and machine learning. The training data is then loaded, and summary statistics and visuals are created to better understand it. The data is split into training and validation sets to check the model's performance.

After that the data is normalized, missing values and outliers will be fixed, and the most important features are chosen to focus on key information. Machine learning models are set up, and their settings are adjusted before training on the training data. The models are then tested on the validation set to see how well they work.

Hyperparameter tuning is done using Optuna, running a loop to find the best setup. Performance metrics are used to check how accurate and effective the models are, and the results are compared to find the best model.

The top model will then be retrained using the entire training dataset. The test dataset is loaded, and predictions are made using the trained model. These predictions are saved in a submission file format. The trained model is exported for future use, and visualizations are generated for further analysis. The process concludes with a finalized model, predictions, and insights ready for deployment or further study.

RESULTS AND DISCUSSION

1. Program Implementation

```
train = pd.read_csv(r"C:\Users\Witssikun\Desktop\Cpe-Cp2-VolcanicEruptionPrediction\train.csv")
sample_submission = pd.read_csv(r"C:\Users\Witssikun\Desktop\Cpe-Cp2-VolcanicEruptionPrediction\submission.csv")
```

```
import numpy as np
import pandas as pd
import glob
import math

import plotly.express as px
from plotly.subplots import make_subplots
import plotly.graph_objs as go

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error as mse
from sklearn.feature_selection import RFE
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeRegressor

from lightgbm import LGBMRegressor
from xgboost import XGBRegressor

import optuna
from optuna.samplers import TPESampler

import tensorflow as tf
import tensorflow.keras.backend as K
from tensorflow.keras.layers import Input, Dense
from sklearn.model_selection import KFold
from tensorflow.keras.callbacks import EarlyStopping
```

```
fig = px.histogram(
    train,
    x="time_to_eruption",
    width=800,
    height=500,
    nbins=100,
    title='Time to eruption distribution'
)
fig.show()
```

```
fig = px.line(
    train,
    y="time_to_eruption",
    width=800,
    height=500,
    title='Time to eruption for all volcanos'
)
fig.show()
```

```
train['time_to_eruption'].describe()
```

```
print('Median:', train['time_to_eruption'].median())
print('Skew:', train['time_to_eruption'].skew())
print('Std:', train['time_to_eruption'].std())
print('Kurtosis:', train['time_to_eruption'].kurtosis())
print('Mean:', train['time_to_eruption'].mean())
```

Figure 1. Data Overview

In figure 1 This line of cells introduces the importing required and the analyzing of the datasets needed to fully utilize this project. By successfully implementing the code and the data sets we can now proceed to training and the testing of fragments.

```
import os
sensors = set()
observations = set()
nan_columns = list()
missed_groups = list()
for_df = list()

for item in train_frags:
    name = int(os.path.splitext(os.path.basename(item))[0])
    at_least_one_missed = 0
    missed_group = list()
    missed_percents = []

    frag = pd.read_csv(item)

    # Only consider the first 10 columns for sensors
    for col in frag.columns[:10]: # Adjust for your sensor columns
        missed_percent = frag[col].isnull().sum() / len(frag)
        missed_percents.append(missed_percent)

        if pd.isnull(frag[col]).all():
            at_least_one_missed = 1
            nan_columns.append(col)
            missed_group.append(col)

    if len(missed_group) > 0:
        missed_groups.append(missed_group)

    sensors.add(len(frag.columns))
    observations.add(len(frag))

    row = [name, at_least_one_missed] + missed_percents[:10]
    if len(row) != 12:
        print(f"Row length mismatch: {len(row)} values. Row: {row}")
        continue # Skip problematic rows

    for_df.append(row)

print(f"Total rows in for_df: {len(for_df)}")
print(f"Sample row lengths: {[len(row) for row in for_df[:5]]}")
```

```

fig = make_subplots(rows=1, cols=2)
traces = [
    go.Histogram(
        x=train[train['has_missed_sensors']==1]['time_to_eruption'],
        nbins=100,
        name="has missed sensors"
    ),
    go.Histogram(
        x=train[train['has_missed_sensors']==0]['time_to_eruption'],
        nbins=100,
        name="Doesn't have missed sensors"
    )
]

for i in range(len(traces)):
    fig.append_trace(
        traces[i],
        (i // 2) + 1,
        (i % 2) + 1
    )

fig.update_layout(
    title_text="Time to eruption distribution for segments with / without missed sensors",
    height=600,
    width=1000
)
fig.show()

```

```

sensors = set()
observations = set()
nan_columns = list()
missed_groups = list()
for test_df in test_frags:
    # Use os.path for file name parsing
    name = int(os.path.splitext(os.path.basename(item))[0])
    at_least_one_missed = 0
    missed_group = list()
    missed_percents = []

    # Read the Fragment CSV
    frag = pd.read_csv(item)

    # Process only the first 10 sensors (adjust as needed)
    for col in frag.columns[:10]:
        missed_percent = frag[col].isnull().sum() / len(frag)
        missed_percents.append(missed_percent)

        if pd.isnull(frag[col]).all():
            at_least_one_missed = 1
            nan_columns.append(col)
            missed_group.append(col)

    if missed_group:
        missed_groups.append(missed_group)

    sensors.add(len(frag.columns))
    observation.add(len(frag))

    # Ensure row has a consistent length
    row = [name, at_least_one_missed + missed_percents[:10]]
    if len(row) != 12:
        print(f"Row length mismatch: {len(row)}. Row: {row}")
        continue

    for test_df.append(row)

# Debugging output
print(f"Processed {len(for_test_df)} fragments.")
print(f"Sample row lengths: {[len(row) for row in for_test_df[:5]]}")

```

```

try:
    for_test_df = pd.DataFrame(
        for_test_df,
        columns=[
            'segment_id', 'has_missed_sensors', 'missed_percent_sensor1',
            'missed_percent_sensor2', 'missed_percent_sensor3', 'missed_percent_sensor4',
            'missed_percent_sensor5', 'missed_percent_sensor6', 'missed_percent_sensor7',
            'missed_percent_sensor8', 'missed_percent_sensor9', 'missed_percent_sensor10'
        ]
    )
except ValueError as e:
    print(f"Error creating DataFrame: {e}")

# Display the DataFrame
for_test_df

```

```

fig = make_subplots(rows=5, cols=2)
traces = [
    go.Scatter(
        x=[i for i in range(60002)],
        y=check[col],
        mode='lines',
        name=col
    ) for col in check.columns
]

for i in range(len(traces)):
    fig.append_trace(
        traces[i],
        (i // 2) + 1,
        (i % 2) + 1
    )

fig.update_layout(
    title_text="Data from sensors",
    height=800,
    width=1200
)

fig.show()

```

Figure 2. Train and test fragments

This figure is important in machine learning workflows, especially for the project's task of predicting volcanic eruptions. The train part gives data to the machine learning model to help it learn patterns. During this stage, the model adjust its settings to reduce mistakes. The Test section, on the other hand, checks how well the model works with new, unseen data. It's a separate part of

the dataset with only input features, either without labels or with labels saved for testing. Both parts are needed to stop the model from overfitting.

```

def build_features(signal, ts, sensor_id):
    X = pd.DataFrame()
    f = np.fft.fft(signal)
    f_real = np.real(f)

    X.loc[ts, f'(sensor_id)_sum'] = signal.sum()
    X.loc[ts, f'(sensor_id)_mean'] = signal.mean()
    X.loc[ts, f'(sensor_id)_std'] = signal.std()
    X.loc[ts, f'(sensor_id)_var'] = signal.var()
    X.loc[ts, f'(sensor_id)_max'] = signal.max()
    X.loc[ts, f'(sensor_id)_min'] = signal.min()
    X.loc[ts, f'(sensor_id)_skew'] = signal.skew()
    X.loc[ts, f'(sensor_id)_mad'] = signal.mad()
    X.loc[ts, f'(sensor_id)_kurtosis'] = signal.kurtosis()
    X.loc[ts, f'(sensor_id)_quantile90'] = np.quantile(signal, 0.99)
    X.loc[ts, f'(sensor_id)_quantile95'] = np.quantile(signal, 0.95)
    X.loc[ts, f'(sensor_id)_quantile85'] = np.quantile(signal, 0.85)
    X.loc[ts, f'(sensor_id)_quantile75'] = np.quantile(signal, 0.75)
    X.loc[ts, f'(sensor_id)_quantile55'] = np.quantile(signal, 0.55)
    X.loc[ts, f'(sensor_id)_quantile45'] = np.quantile(signal, 0.45)
    X.loc[ts, f'(sensor_id)_quantile25'] = np.quantile(signal, 0.25)
    X.loc[ts, f'(sensor_id)_quantile15'] = np.quantile(signal, 0.15)
    X.loc[ts, f'(sensor_id)_quantile05'] = np.quantile(signal, 0.05)
    X.loc[ts, f'(sensor_id)_quantile01'] = np.quantile(signal, 0.01)
    X.loc[ts, f'(sensor_id)_fft_real_mean'] = f_real.mean()
    X.loc[ts, f'(sensor_id)_fft_real_std'] = f_real.std()
    X.loc[ts, f'(sensor_id)_fft_real_max'] = f_real.max()
    X.loc[ts, f'(sensor_id)_fft_real_min'] = f_real.min()

    return X

```

```

sampler = TPEsampler(seed=666)

def create_model(trial):
    num_leaves = trial.suggest_int('num_leaves', 2, 31)
    n_estimators = trial.suggest_int('n_estimators', 50, 300)
    max_depth = trial.suggest_int('max_depth', 2, 8)
    min_child_samples = trial.suggest_int('min_child_samples', 100, 1200)
    learning_rate = trial.suggest_uniform('learning_rate', 0.0001, 0.99)
    min_data_in_leaf = trial.suggest_int('min_data_in_leaf', 5, 90)
    bagging_fraction = trial.suggest_uniform('bagging_fraction', 0.0001, 1.0)
    feature_fraction = trial.suggest_uniform('feature_fraction', 0.0001, 1.0)
    model = LGBMRegressor(
        num_leaves=num_leaves,
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_child_samples=min_child_samples,
        min_data_in_leaf=min_data_in_leaf,
        learning_rate=learning_rate,
        bagging_fraction=feature_fraction,
        random_state=666
    )
    return model

def objective(trial):
    model = create_model(trial)
    model.fit(train, y)
    preds = model.predict(val)
    score = rmse(y_val, preds)
    return score

# To use optuna uncomment it
# study = optuna.create_study(direction="minimize", sampler=sampler)
# study.optimize(objective, n_trials=5000)
# params = study.best_params
# params['random_state'] = 666

params = {
    'num_leaves': 29,
    'n_estimators': 289,
    'max_depth': 8,
    'min_child_samples': 507,
    'learning_rate': 0.00126327662599,
    'min_data_in_leaf': 13,
    'bagging_fraction': 0.020521665677937423,
    'feature_fraction': 0.05776459074779527,
    'random_state': 666
}

lgb = LGBMRegressor(**params)
lgb.fit(train, y)

```

```

params = {
    'max_depth': 6,
    'n_estimators': 189,
    'learning_rate': 0.00910718143795864,
    'gamma': 0.787986320220815,
    'random_state': 666
}

xgb_short = XGBRegressor(**params)

xgb_short.fit(reduced_train, reduced_y)
preds = xgb_short.predict(reduced_val)
print('XGBoost rmse', rmse(reduced_y_val, preds))

```

```

def root_mean_squared_error(y_true, y_pred):
    return K.sqrt(K.mean(K.square(y_pred - y_true), axis=-1))

def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input((24,)),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.Dense(1000, activation='sigmoid'),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.Dropout(0.6),
        tf.keras.layers.Dense(1, activation='relu')
    ])

    model.compile(
        loss=root_mean_squared_error,
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001)
    )
    return model

```

```

models = list()
preds = list()

for n, (tr, te) in enumerate(kfold(
    X_split=X,
    y_split=y,
    shuffle=True,
    shuffle_seed=seed)):

    early_stopping = EarlyStopping(
        patience=10,
        verbose=0
    )

    print(f'Fold {n}')

    model = create_model()

    model.fit(
        train_values[tr],
        yy_values[tr],
        verbose=0,
        batch_size=128,
        verbose=0,
        callbacks=[early_stopping]
    )

    pred = model.predict(val)
    pred = np.exp1(pred).reshape((pred.shape[0], ))
    preds.append(pred)
    print('RMSE train, rmse (yy_values[te], model.predict(train_values[te]))')
    models.append(model)

predictions = preds[0]
for i in range(1, 3):
    predictions += preds[i]
predictions /= 3

print('RM rmse', rmse(y_val, predictions))

test_set = list()
for seg in sample_submission.segment_id:
    signals = pd.read_csv(f'/kaggle/input/predict-volcanic-eruptions-ingr-oe/test/{seg}.csv')
    test_row = []
    if JK000 == 0:
        print()
        for i in range(0, 10):
            sensor_id = f'sensor_{i+1}'
            test_row.append(build_features(signals[sensor_id].fillna(0), seg, sensor_id))
        test_row = pd.concat(test_row, axis=1)
        test_set.append(test_row)
    else:
        test_set = pd.concat(test_set)

```

Figure 3. Modeling

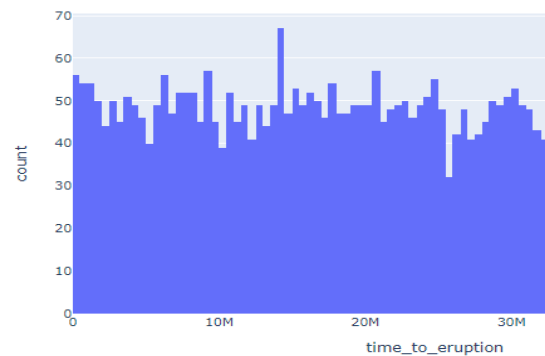
In this Figure, it is the section where machine learning models are built, trained, evaluated and optimized. The model selection chooses one or more machine algorithms to use for prediction. Models like **LightGBM**, **XGBoost**, or **TensorFlow/Keras** are often defined in this step. Defining Model Architecture sets up the parameters and structure of the chosen models. For **tree-based models** (e.g., LightGBM, XGBoost), parameters like the number of trees, learning rate, and maximum depth are defined. Training Models fit the models to the training dataset (features and labels) into the model. Validation assesses the model's performance on validation data to ensure it is not overfitting. Metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), or R^2 are calculated to evaluate performance. Hyperparameter Tuning optimizes the model by finding the best set of hyperparameters. Tools like **Optuna** or **GridSearchCV** are used to systematically explore different combinations of hyperparameters (e.g., learning rate, number of estimators). Model Comparison compares the performance of multiple models to select the best one. Metrics from different models (e.g., LightGBM vs. XGBoost) are compared to determine which performs best. Final Model Training retrains the selected best model using the full training dataset. Prediction on Test Data uses the trained model to make predictions on the test dataset. Performance Visualization analyzes the model's performance using plots or visualizations. The **modeling part of the code** involves defining, training,

evaluating, tuning, and selecting models to achieve the best possible predictive performance. This phase is critical for ensuring the model's accuracy, generalization capability, and suitability for the volcanic eruption prediction task.

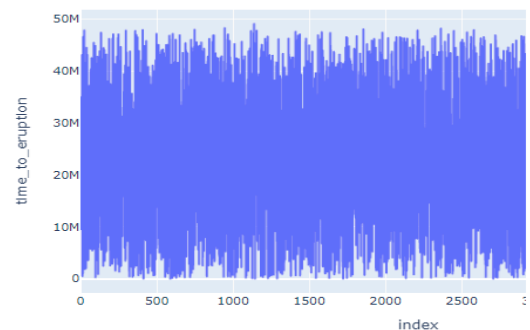
II. Program Output

	segment_id	time_to_eruption
0	1136037770	12262005
1	1969647810	32739612
2	1895879680	14965999
3	2068207140	26469720
4	192955606	31072429
---	---	---
4426	873340274	15695097
4427	1297437712	35659379
4428	694853998	31206935
4429	1886987043	9598270
4430	1100632800	20128938

Time to eruption distribution



Time to eruption for all volcanos



```

count      4.431000e+03
mean       2.284891e+07
std        1.348439e+07
min        6.250000e+03
25%        1.127016e+07
50%        2.246559e+07
75%        3.434356e+07
max        4.904609e+07
Name: time_to_eruption, dtype: float64

```

```

Median: 22465586.0
Skew: 0.059347778798359965
Std: 13484385.863632679
Kurtosis: -1.166601027916375
Mean: 22848906.832769126

```

Figure 4. Data Overview Results

This result displays time-series data for each of the volcanic samples. These signals are classified into several categories, highlighting how volcanoes evolve throughout time. Along with the time-series data, the graphic includes summary statistics to assist us better comprehend the data. This provides information on the distribution of values, the range of data points, and any distinctive patterns or outliers. Some attributes may have unequal distributions or vast ranges, necessitating normalization or transformation of the data to facilitate analysis and comparison.

	sensor_1	sensor_2	sensor_3	sensor_4	sensor_5	sensor_6	sensor_7
0	-343.0	-110.0	-274.0	367.0	137.0	-106.0	-578.0
1	-298.0	-92.0	-159.0	288.0	76.0	-251.0	-713.0
2	-503.0	195.0	-140.0	266.0	-100.0	-162.0	-450.0
3	-153.0	-68.0	-78.0	301.0	-143.0	-86.0	-378.0
4	-320.0	-348.0	-27.0	283.0	-26.0	-255.0	-595.0
...	...	...	...	...	...	...	...
59996	329.0	-1.0	573.0	155.0	76.0	108.0	-727.0
59997	-125.0	32.0	599.0	243.0	99.0	105.0	-788.0
59998	403.0	17.0	555.0	360.0	98.0	-66.0	-773.0
59999	-212.0	-92.0	548.0	163.0	71.0	-13.0	-207.0
60000	-131.0	-179.0	497.0	223.0	139.0	81.0	98.0

Figure 5.

The data from 10 sensors and 60001 observations for each of them.

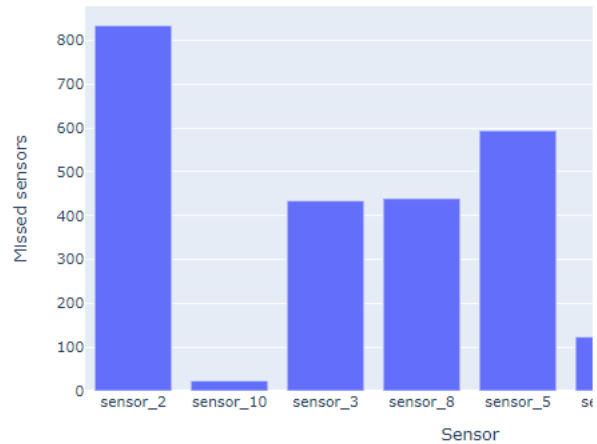


Figure 6.

Figure 6 shows that the output don't have missed sensors for all volcanoes for sensor 4 and sensor 6 in the training set.

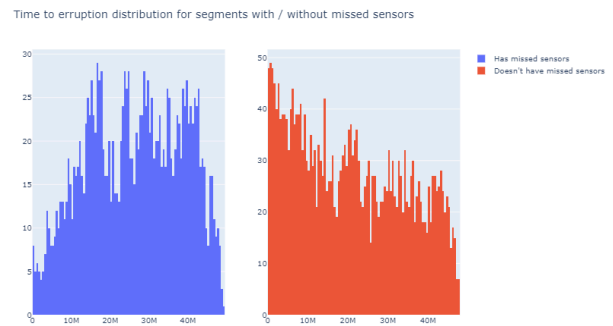


Figure 7.

This output shows the time to eruption segments with and without missed sensors.



Figure 8.

The data from all sensors for the current volcano.


```

Processing segment: 0
Processing segment: 500
Processing segment: 1000
Processing segment: 1500
Processing segment: 2000
Processing segment: 2500
Processing segment: 3000
Processing segment: 3500
Processing segment: 4000
Successfully created train_set with 4431 rows.

```

Figure 9.

In this figure, the output is a DataFrame called *train_set*, which contains the mean values of the sensor data from each segment of volcanic eruption prediction data.

	segment_id	sensor_1_sum	sensor_1_mean	sensor_1_std	sensor_1_var
0	1136037770	-96621.0	-1.610323	303.096099	91867.245099
1	1969647810	85569.0	1.426126	438.360560	192159.98041
2	1895879680	150278.0	2.504592	241.575415	58358.681050
3	2068207140	129950.0	2.165797	221.967825	49269.715377
4	192955606	4429.0	0.073815	261.695935	68484.762635
...	...	...	...	...	...
4426	873340274	54405.0	0.906735	613.427301	376293.05413
4427	1297437712	476221.0	7.936884	649.089618	421317.33273
4428	694853998	85261.0	1.420993	110.942596	12308.259562
4429	1886987043	54350.0	0.905818	478.852581	229299.79401
4430	1100632800	-248575.0	-4.142848	272.632730	74328.605478

Figure. 10

The result, as shown in the picture, is a combined DataFrame with a column entitled *segment_id*, which reflects each segment's unique identification. It also contains sensor data from the *train_set*.

CONCLUSION

[4] This project focuses on predicting volcanic eruptions using geophysical data and machine learning. Volcanic eruptions are dangerous events that can cause loss of life, damage to property, and harm to the environment. Predicting these events accurately and on time is important to help communities prepare and reduce the damage they cause. [5] The research starts by studying patterns and relationships in data such as seismic activity, acoustic signals, gas emissions, and ground movement. [6] Using exploratory data analysis (EDA), it identifies key factors that can signal an eruption.

[7] To improve prediction accuracy, the project transforms raw data into useful features like averages, standard deviations, and other simple measures. [8] These features are used to train machine learning models,

including advanced algorithms like Gradient Boosted Trees (e.g., XGBoost and LightGBM). These models are fine-tuned and tested to ensure they give reliable predictions.[9] Their performance is measured using metrics like Mean Squared Error (MSE) and R-squared (R^2), while feature importance analysis helps identify which factors are most important for making predictions.

The main result of the project is a prediction system that can estimate the chance and timing of volcanic eruptions. This system provides timely and helpful information that allows governments, emergency teams, and communities to prepare and respond better to potential eruptions. By doing so, it can save lives, reduce damage to property, and minimize the disruption caused by volcanic activity.

This research also shows how advanced technology can improve disaster preparedness. [10] By combining machine learning with existing monitoring systems, it offers a new way to understand and respond to natural hazards. While the focus is on volcanic eruptions, the same approach could be used to address other natural disasters like earthquakes and hurricanes.

In the end, this project is about using technology to help people. It aims to create tools that protect lives and make communities safer. By focusing on practical solutions and working with others, the research offers hope for reducing the risks and impacts of natural disasters. This work is a step toward a future where communities can better prepare for and respond to the challenges posed by nature.

RECOMMENDATIONS

To make this project truly impactful, the machine learning model should be integrated into real-time volcanic monitoring systems. By analyzing live data such as seismic activity, gas emissions, and ground movements it can provide timely and accurate warnings of potential eruptions. Collaborating with organizations like PHIVOLCS or international volcanic monitoring agencies will ensure the model has access to the most up-to-date information. This kind of partnership will help bring the system out of the lab and into real-world use, where it can play a critical role in protecting lives and property.

The data used for the model should also be as diverse and comprehensive as possible. Volcanoes behave differently depending on their location and type, so including data from a wide range of settings will make the predictions more reliable. Adding satellite imagery and remote sensing tools can improve the system's ability to monitor remote or hard-to-reach volcanoes. Testing the model in real-world conditions over an extended period

will also be important for refining its performance and ensuring it can handle the complexities of actual volcanic activity. These steps will make the system stronger and better prepared for real-world challenges.

Finally, it's crucial to focus on public awareness and preparedness. Communities near volcanoes need to understand how the prediction system works and what to do when an alert is issued. Clear communication can prevent panic and ensure people know how to respond. Training for disaster response teams will also help ensure that warnings lead to effective actions on the ground. By fostering collaboration between scientists, local authorities, and community leaders, this project can become more than just a technological solution—it can be a life-saving tool that helps people stay safe in the face of unpredictable natural events. Regular updates and improvements will keep the system accurate and reliable for years to come.

REFERENCES

- [1] Tomar, J. (2024, September 20). Top 9 Data Science Projects with Source Code [Updated]. *GUVI Blogs*. <https://www.guvi.in/blog/data-science-projects-with-source-code/>
- [2] Sparks, R. S. J. (2003). Forecasting volcanic eruptions. *Earth and Planetary Science Letters*, 210(1–2), 1–15. [https://doi.org/10.1016/S0012-821X\(03\)00124-9](https://doi.org/10.1016/S0012-821X(03)00124-9)
- [3] BUHL, N. (2024, DECEMBER 10). TRAINING, VALIDATION, TEST SPLIT FOR MACHINE LEARNING DATASETS. [HTTPS://ENCORD.COM/BLOG/TRAIN-VAL-TEST-SPLIT/](https://encord.com/blog/train-val-test-split/)
- [4] Barberi, F., Corrado, G., Innocenti, F., & Luongo, G. (1984). Phlegraean Fields 1982–1984: Brief chronicle of a volcano emergency in a densely populated area. *Bulletin Volcanologique*, 47(2), 175–185. <https://doi.org/10.1007/BF01961547>
- [5] McNutt, S. R. (2005). Volcanic seismology. *Annual Review of Earth and Planetary Sciences*, 33, 461–491. <https://doi.org/10.1146/annurev.earth.33.092203.122459>
- [6] Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- [7] Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- [8] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- [9] Powers, D. M. W. (2011). Evaluation: From precision, recall, and F-measure to ROC, informedness, markedness & correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63.
- [10] Weston, P., Blackett, M., & Montserrat, S. (2021). Machine learning applications in volcano monitoring. *Journal of Volcanology and Geothermal Research*, 414, 107210. <https://doi.org/10.1016/j.jvolgeores.2021.107210>
- [11] Dzurisin, D. (2007). *Volcano deformation: Geodetic monitoring techniques*. Springer.
- [12] Weston, P., Blackett, M., & Montserrat, S. (2021). Machine learning applications in volcano monitoring. *Journal of Volcanology and Geothermal Research*, 414, 107210. <https://doi.org/10.1016/j.jvolgeores.2021.107210>
- [13] Sparks, R. S. J., Aspinall, W. P., Crosweller, H. S., & Hincks, T. K. (2013). Risk and uncertainty assessment for natural hazards. *Risk Analysis*, 33(4), 640–650. <https://doi.org/10.1111/j.1539-6924.2012.01873.x>

ABOUT THE AUTHORS



Eurylyn Florido is a Second Year College Student At FAITH Colleges taking Bachelor of Science in Computer Engineering . She is currently the Head DCO of the Faith Computer Engineering Society.



John Mitchell Fruto is a Second Year College Student at FAITH Colleges taking Bachelor of Science in Computer Engineering . He is currently the DCO of the Faith Computer Engineering Society.